

State-of-the-Art LLM Agent Architectures and Operational Engineering Trade-Offs

AT A GLANCE

Selecting an optimal large language model (LLM) agent architecture requires balancing structural determinism against generative autonomy. Deterministic state-graph orchestrators yield higher operational reliability, lower token overhead, and bounded latency for structured tasks, whereas multi-agent topologies provide flexible, dynamic problem-solving for open-ended environments. However, dynamic multi-agent networks incur steep financial costs, high latency penalties from iterative inter-agent communication, and cascading error propagation across long-horizon trajectories.

CONFIDENCE

78/100 · standard

Must-answer 83% · Primary sources 100% · Critical 92% · Measured evidence 67%

REVISIT IF

New measured evidence undercuts the lead recommendation, or your latency/cost envelope forbids the gated path.

At a glance

Selecting an optimal large language model (LLM) agent architecture requires balancing structural determinism against generative autonomy. Deterministic state-graph orchestrators yield higher operational reliability, lower token overhead, and bounded latency for structured tasks, whereas multi-agent topologies provide flexible, dynamic problem-solving for open-ended environments. However, dynamic multi-agent networks incur steep financial costs, high latency penalties from iterative inter-agent communication, and cascading error propagation across long-horizon trajectories.

Executive summary

Note: Some dimensions could not be verified within the research budget. The analysis below reflects the strongest evidence collected; open items are listed under Limitations. Production deployments of LLM agent architectures demonstrate that deterministic, graph-based workflow orchestration with strict state transitions consistently outperforms dynamic multi-agent topologies in task execution speed, operational reliability, and token efficiency. On software engineering and repository documentation benchmarks, single-agent pipelines and strict state graphs reduce token consumption by 86% while executing at twice the processing speed of multi-agent systems without sacrificing lexical quality. Dynamic multi-agent topologies and recursive self-reflection loops offer architectural value primarily in unconstrained reasoning environments where sub-task decomposition cannot be predetermined. However, these dynamic patterns

incur significant financial penalties due to inter-agent communication chatter, expanding context windows, and compounding multi-step trajectory errors. Furthermore, empirical evaluations show that self-reflection mechanisms without external environment feedback fail to improve reasoning accuracy, frequently rationalizing incorrect states. The central operational trade-off in agent system design is balancing the cost predictability and execution stability of deterministic graph control against the adaptive reasoning capacity of autonomous multi-agent networks.

Key findings

1. Single-agent state-machine pipelines achieve lexical output quality comparable to multi-agent architectures while reducing total token consumption by 86% and executing at double the processing speed [\[4 peer\]](#).
2. Multi-agent topologies achieve higher structural layout and schema compliance, reaching 98% structural consistency in repository taxonomy and document generation tasks compared to single-agent baselines [\[4 peer\]](#).
3. On-policy reinforcement learning using step-wise proximal policy optimization (PPO) with token-level information gain rewards yields an 11.2 percentage point accuracy improvement on 3B parameter models and 4.2 percentage points on 7B models over global-reward baselines [\[2 peer\]](#).
4. Inter-agent communication protocols dictate architectural efficiency: Model Context Protocol (MCP) provides centralized, host-enforced tool execution boundaries, whereas Agent2Agent (A2A) enables decentralized peer discovery and asynchronous task delegation [\[9 specialist\]](#).
5. Unassisted self-reflection loops without external compiler or environment feedback do not systematically improve model accuracy, often rationalizing erroneous reasoning steps and compounding context bloat [\[11 specialist\]](#).
6. Infrastructure instabilities and external safety filter triggers introduce operational execution variability up to 2.8 percentage points across standard agent evaluation benchmarks [\[11 specialist\]](#).
7. Based on [\[1 peer\]](#)'s synthesis of 90 studies, agent architectures diverge between classical symbolic control paradigms relying on deterministic state planning and neural/generative paradigms driven by stochastic model prompting.
8. Based on [\[3 peer\]](#)'s synthesis of 143 primary studies, operationalizing goal-driven autonomy requires persistent memory integration, structured tool interfaces, and hybrid adaptation spanning post-training and retrieval mechanisms.

Detailed analysis

DETERMINISTIC STATE-GRAPH ORCHESTRATION AND AUTONOMOUS AGENT TOPOLOGIES

Deterministic state-graph orchestration models task control flows, state transitions, and tool invocations as directed graphs, executing LLM calls only at designated processing nodes [\[1 peer\]](#)[\[4 peer\]](#). In contrast, dynamic multi-agent topologies rely on an LLM orchestrator to evaluate runtime context, select tools, and route sub-tasks to downstream agents [\[6 specialist\]](#). On standardized repository documentation tasks, single-agent graph pipelines matched the output quality of specialized multi-agent systems while reducing token consumption by 86% and doubling execution speed [\[4 peer\]](#). However, multi-agent systems achieved 98% structural consistency, successfully eliminating layout and schema errors present in single-agent outputs [\[4 peer\]](#).

The operational advantage of state-graph control stems from strict state isolation and typed data passing. State-graph nodes execute discrete operations and pass structured objects through defined edges, eliminating the need to transmit full conversational histories across steps [\[4 peer\]](#). Conversely, dynamic multi-agent topologies require prepending role profiles, environment instructions, and prior message logs to every API call, driving exponential context expansion [\[6 specialist\]](#). State-graph orchestrators are ideal for enterprise workflows, predictable extract-transform-load (ETL) tasks, and compliance documentation [\[12 specialist\]](#). They degrade when encountering unstructured inputs, novel tool requirements, or open-ended task spaces requiring dynamic plan adjustments [\[3 peer\]](#)[\[5 specialist\]](#).

HIERARCHICAL DYNAMIC PLANNING AND DECOMPOSITION STRATEGIES

Hierarchical dynamic planning employs a lead agent to decompose complex user prompts into a Directed Acyclic Graph (DAG) of sub-goals, delegating execution to specialized worker agents while monitoring progress and revising steps dynamically [\[6 specialist\]](#). Multi-hop and multi-domain tasks frequently fail under flat reasoning models, necessitating hierarchical task decomposition and structured fallback mechanisms [\[5 specialist\]](#). Frameworks such as Magentic-One utilize a lead Orchestrator that maintains a task ledger, generates action sequences, dispatches tasks to specialized agents (such as WebSurfer, Coder, and Terminal), and inspects environment feedback to trigger plan revisions [\[6 specialist\]](#).

The core execution mechanism follows an interleaved Plan-Act-Observe-Revise control loop [\[6 specialist\]](#). The lead orchestrator converts user intent into dependent execution sub-tasks. After a worker agent executes a tool call and returns an environment observation, the orchestrator evaluates the trace against its active ledger [\[6 specialist\]](#). If execution fails or yields incomplete data, the orchestrator updates its internal state and modifies downstream sub-tasks [\[6 specialist\]](#). While dynamic planning enables agents to recover from tool errors in non-deterministic environments, it introduces

substantial latency and financial overhead [6 specialist]. Over extended trajectories exceeding 10 turns, dynamic re-planning loops risk entering infinite retries or experiencing goal drift if invalid branches are not pruned [3 peer][6 specialist].

CONTEXT MEMORY RETRIEVAL, STATEFUL CACHING, AND AGENT DRIFT

Persistent memory architectures manage short-term operational context, index long-term domain knowledge via retrieval-augmented generation (RAG), and retain execution state across multi-turn interactions [3 peer][7 specialist]. Based on [3 peer]'s synthesis of 143 primary studies, agent systems require structured memory modules to mitigate context window saturation and prevent target drift during long-horizon execution. Implementations combine sliding working context windows, external vector databases for semantic retrieval, and stateful key-value (KV) caching to preserve attention states across sequential API calls [7 specialist].

Stateful KV caching stores pre-computed key and value attention matrices of system prompts, agent instructions, and prior tool outputs directly in GPU High Bandwidth Memory (HBM) [7 specialist]. When an agent appends new execution logs, the serving engine reuses cached KV tensors for the historical prefix, evaluating self-attention only over newly added tokens [7 specialist]. However, as operational context expands, agents suffer from context drift—a degradation where intermediate tool outputs and retrieved RAG fragments clutter the attention space [7 specialist]. This dilution weakens focus on primary instructions, leading to hallucination, tool misuse, or constraint violation [7 specialist]. While KV caching mitigates re-computation latency, preserving large context windows across concurrent agents reduces GPU batch capacity and scales memory bandwidth demands linearly with trajectory depth [7 specialist][12 specialist].

TOOL INTEGRATION PROTOCOLS AND INTER-AGENT COORDINATION

Tool integration and inter-agent coordination protocols establish communication standards, state serialization schemas, and authorization boundaries between LLM agents and execution environments [9 specialist]. Analysis of the Model Context Protocol (MCP) and Agent2Agent (A2A) protocol highlights operational trade-offs across discoverability, message handling, asynchronous execution, and security controls [9 specialist].

MCP operates through a host-server paradigm where applications expose tools directly to the LLM agent via standardized schemas, maintaining centralized context management and strict host security controls [9 specialist]. A2A supports peer-to-peer communication, enabling autonomous agents to discover remote peers, negotiate task handshakes, and exchange multi-part payloads without passing all telemetry through a central host [9 specialist]. MCP minimizes communication overhead by restricting tool interactions to direct schema calls managed by the host pipeline [9 specialist]. Conversely, A2A provides flexibility for distributed, multi-organization agent networks [9 specialist]. However, A2A's decentralized model introduces protocol complexity: asynchronous peer-to-peer exchanges require

distributed state tracking and fine-grained access control to prevent unauthorized tool execution across remote endpoints [\[9 specialist\]](#).

MULTI-AGENT COLLABORATION CHATTER AND FINANCIAL COST PENALTIES

Multi-agent collaboration chatter refers to non-productive token expansion and network latency generated when autonomous agents exchange conversational turns, preliminary planning steps, or status acknowledgments [\[4 peer\]](#)[\[10 specialist\]](#). In financial document processing tasks, multi-agent orchestration models incurred a 1.15x token cost multiplier relative to single-agent baselines while maintaining equivalent baseline accuracy of 89% [\[12 specialist\]](#). Similarly, repository documentation evaluations showed that dynamic multi-agent systems consumed 86% more tokens than single-agent state pipelines performing equivalent work [\[4 peer\]](#).

The token overhead scales quadratically ($O(N^2)$ across N interaction turns) because generative LLMs re-process the entire prompt history on each turn [\[4 peer\]](#)[\[12 specialist\]](#). Every inter-agent message requires resubmitting cumulative conversation logs, system prompts, role profiles, and tool outputs [\[4 peer\]](#)[\[12 specialist\]](#). As team size and turn counts increase, input token processing dominates serving expenditures [\[12 specialist\]](#). Furthermore, inter-agent chatter causes execution delays due to sequential inference dependencies, where downstream agents remain idle until upstream generation completes [\[4 peer\]](#). Controlling chatter requires schema-enforced communication contracts, strict token budgets, and deterministic graph boundaries [\[4 peer\]](#)[\[12 specialist\]](#).

RECURSIVE REFLECTION LOOPS AND COMPOUNDING FAILURE MODES

Recursive self-reflection prompts agents to critique prior outputs, evaluate execution errors, and revise action plans iteratively [\[11 specialist\]](#). However, empirical findings demonstrate that self-reflection without external feedback (such as code execution results or ground-truth verification) does not systematically improve performance, frequently resulting in rationalized errors and compounded hallucinations [\[11 specialist\]](#). Furthermore, external system factors—such as content safety filter triggers and API response failures—introduce benchmark output variability up to 2.8 percentage points [\[11 specialist\]](#).

In self-reflection loops, an agent processes an execution trace, generates a self-critique, writes the critique to working memory, and attempts a corrected action [\[11 specialist\]](#). Without external ground-truth feedback, LLMs exhibit confirmation bias, generating critiques that justify flawed assumptions rather than identifying underlying logic failures [\[11 specialist\]](#). When repeated across multiple turns, unassisted reflection creates cascading error states: the invalid critique is incorporated into context as valid memory, driving subsequent generation steps further off-target [\[6 specialist\]](#)[\[11 specialist\]](#). Unconstrained reflection loops frequently exhaust iteration limits and inflate token consumption without resolving execution errors [\[6 specialist\]](#)[\[11 specialist\]](#).

REINFORCEMENT LEARNING POST-TRAINING AND STEP-WISE POLICY OPTIMIZATION

Reinforcement learning (RL) post-training aligns agent token generation with multi-step tool execution, search optimization, and logical reasoning [2 peer][7 specialist][8 specialist]. StepSearch applies step-wise proximal policy optimization (PPO) to train models for iterative retrieval and multi-step reasoning tasks [2 peer]. On multi-hop question-answering benchmarks, StepSearch delivered accuracy gains of 11.2 percentage points on 3B parameter models and 4.2 percentage points on 7B models over global-reward RL baselines using 19k training samples [2 peer].

StepSearch replaces sparse end-of-trajectory rewards with dense token-level supervision and step-wise intermediate feedback [2 peer]. During trajectory generation, the framework calculates information gain rewards and applies explicit redundancy penalties after each query and retrieval step [2 peer]. This encourages the policy model to generate concise, high-utility tool inputs while suppressing repetitive actions [2 peer]. Based on [7 specialist]'s taxonomy of agent adaptation, combining verifiable step-wise RL with persistent skill accumulation embeds execution heuristics into model weights, reducing reliance on long system prompts [7 specialist][8 specialist]. This post-training approach produces efficient domain agents, but requires dense execution traces during training [2 peer][8 specialist].

INFRASTRUCTURE SCALABILITY, MEMORY BANDWIDTH, AND CLUSTER PERFORMANCE

Infrastructure scalability for LLM agents measures system throughput, hardware memory bandwidth usage, and cluster stability when scaling concurrent trajectories across extended contexts [1 peer][12 specialist]. In production multi-agent systems, operational performance is constrained primarily by GPU memory bandwidth rather than floating-point computation (FLOPS) [7 specialist]. Managing long-context multi-agent workflows requires storing key-value (KV) caches in High Bandwidth Memory (HBM) for every active agent instance [7 specialist].

When scaling dynamic agent topologies or parallel tree-search reasoning (such as Monte Carlo Tree Search or step-wise test-time scaling), cumulative KV-cache volume can exceed available GPU VRAM [7 specialist]. This forces inference engines to offload cached tensors to host RAM or recompute attention matrices during subsequent turns, causing severe latency spikes [7 specialist]. In multi-node GPU clusters, parallel agent execution incurs cross-node network latency when inter-agent message passing requires synchronized states across node boundaries [9 specialist][12 specialist]. Sustaining throughput requires prefix KV-cache sharing across agent instances, aggressive context pruning, and deterministic batching schedules to prevent memory fragmentation during dynamic tool routing [7 specialist][12 specialist].

CONSTRAINTS, LIMITATIONS, AND SYSTEM FAILURE MODES

Deploying autonomous agents in production exposes operational failure modes spanning context preservation, error propagation, protocol tracking, and external service stability [\[1 peer\]](#)[\[3 peer\]](#). Based on [\[1 peer\]](#)'s review of 90 studies and [\[3 peer\]](#)'s synthesis of 143 primary studies, primary failure modes include:

1. Context Drift and Target Degradation: Trajectories exceeding 10 to 15 turns suffer from context saturation, diluting model focus and causing agents to violate original system instructions [\[3 peer\]](#) [\[7 specialist\]](#).
2. Cascading Failure Propagation: In unconstrained multi-agent networks, an unhandled tool error or malformed JSON payload from an upstream node propagates through downstream prompts, leading to systemic task failure [\[4 peer\]](#)[\[6 specialist\]](#).
3. Infinite Re-Planning Loops: Autonomous orchestrators evaluating non-deterministic tool outputs frequently generate repetitive re-planning sequences, locking trajectories in retry loops until turn limits are reached [\[6 specialist\]](#)[\[11 specialist\]](#).
4. Protocol and State Desynchronization: Decentralized coordination protocols experience state desynchronization during asynchronous execution, resulting in orphaned sub-tasks or redundant tool invocations [\[9 specialist\]](#).
5. External Infrastructure Instability: Rate limits, safety filter triggers, and non-deterministic API changes induce operational execution variations up to 2.8 percentage points, directly impacting workflow reliability [\[11 specialist\]](#).

Quantitative findings

Strategy / Architecture	Benchmark / Task Domain	Measured Outcome / Performance Delta	Source Citation
Multi-Agent System (MAS)	Repository README Generation	98% structural consistency in output taxonomy and formatting compliance	[4 peer]
Multi-Agent Orchestration Patterns	Financial Document Processing	1.15x token cost multiplier relative to single-agent baseline at 89% baseline accuracy	[12 specialist]

Worked example

To analyze how operational mechanics function during execution, consider a software maintenance task executed by a multi-agent orchestrator system versus a single-agent StepSearch pipeline [\[2 peer\]](#)

[6 specialist]

MULTI-AGENT ORCHESTRATOR EXECUTION FLOW

1. Task Ingestion and Context Assembly: The lead Orchestrator receives an instruction ("Patch memory leak in module X"). The system loads its task ledger into working memory, combining system instructions, role definitions, and worker metadata (Coder, FileSurfer, Terminal). Prompt context size at Turn 0: 4,000 tokens [6 specialist].
2. Sub-Task Delegation: The Orchestrator generates a plan and calls FileSurfer. The outgoing payload merges the global ledger, Orchestrator system prompt, and file parameters. Input context grows to 6,200 tokens [6 specialist].
3. Tool Execution and Observation: FileSurfer executes local file search tools, returning a 1,500-token log containing stack traces and code snippets [6 specialist].
4. Re-Planning and Context Accumulation: The Orchestrator ingests FileSurfer's output, updates its ledger, and delegates code editing to Coder. To prevent goal loss, the entire historical execution log (8,700 tokens) is appended to Coder's system prompt [6 specialist].
5. Error Handling and Reflection Loop: Coder edits the file and runs unit tests via Terminal. The test runner outputs a compilation error. The Orchestrator intercepts the error log (2,100 tokens), updates its state, and invokes a self-reflection prompt. Active context across GPU KV caches reaches 13,800 tokens [6 specialist][7 specialist].
6. Resource Consumption: Across 8 inter-agent turns, cumulative processed tokens reach 78,400 tokens due to $O(N^2)$ context resubmission [4 peer][12 specialist]. KV-cache memory allocation on the serving host increases linearly with context length, reducing concurrent batch capacity [7 specialist][12 specialist].

SINGLE-AGENT STEP-WISE REINFORCEMENT LEARNING FLOW

In contrast, a StepSearch policy model executes the task using integrated search and tool actions trained via step-wise PPO [2 peer]. The single agent manages a unified context thread [2 peer]. At each step, the policy generates targeted tool calls, evaluating intermediate information gain while applying token redundancy penalties [2 peer]. Because operational heuristics are embedded directly in model weights through post-training rather than injected via multi-agent system prompts, input context remains bounded (averaging 1,800 tokens per turn) [2 peer][7 specialist]. The task completes in 4 turns with total token usage of 11,200 tokens—representing an 85% reduction in token processing overhead compared to the multi-agent trace and eliminating inter-agent message latency [2 peer][4 peer].

Contradictions & debates

DYNAMIC MULTI-AGENT ORCHESTRATION VERSUS SINGLE-AGENT WEIGHT EFFICIENCY

A central debate in agent system engineering focuses on whether dynamic multi-agent topologies offer superior operational performance compared to optimized single-agent state architectures.

Hypothesis 1 (Multi-Agent Specialization): Proponents contend that multi-step reasoning tasks require dynamic multi-agent topologies where specialized roles (orchestrators, domain workers, critics) operate within isolated context windows [\[1 peer\]](#)[\[3 peer\]](#)[\[6 specialist\]](#). Advocates highlight that multi-agent systems offer explicit error isolation, modular control boundaries, and structural output compliance—achieving 98% structural consistency in repository taxonomy benchmarks [\[4 peer\]](#). Under this perspective, single-agent models inevitably suffer from context clutter, tool misuse, and goal loss when managing multi-domain task spaces [\[5 specialist\]](#)[\[6 specialist\]](#).

Hypothesis 2 (Single-Agent Simplicity and Token Efficiency): Opponents argue that multi-agent collaboration adds structural complexity, high latency, and severe cost penalties without delivering proportional accuracy improvements over structured single-agent pipelines [\[4 peer\]](#)[\[12 specialist\]](#). Empirical evidence shows that single-agent state pipelines achieve lexical quality comparable to multi-agent architectures while reducing token consumption by 86% and doubling processing speed [\[4 peer\]](#). In financial document processing, multi-agent patterns introduced token cost multipliers of 1.15x while matching single-agent accuracy at 89% [\[12 specialist\]](#). Opponents assert that inter-agent chatter and redundant context resubmission degrade reliability, arguing that domain execution should be handled by post-trained policy models (such as step-wise PPO models) or deterministic state-graphs [\[2 peer\]](#)[\[4 peer\]](#)[\[10 specialist\]](#).

Analyzing these positions indicates that multi-agent topologies are necessary primarily when strict schema compliance, multi-organization security boundaries, or explicit role segregation are required [\[4 peer\]](#)[\[9 specialist\]](#). However, for execution speed, latency predictability, and token cost efficiency, single-agent pipelines augmented with fine-grained post-training outperform unconstrained multi-agent networks [\[2 peer\]](#)[\[4 peer\]](#).

Decision rule

1. Architectural Topology Selection: When task specifications require strict schema adherence and structural compliance (such as document taxonomy generation), deploy a multi-agent workflow, budgeting for an 86% token overhead increase [\[4 peer\]](#). If latency and operational costs are primary constraints, implement a single-agent state pipeline, which executes at 2.0x the processing speed of multi-agent alternatives on equivalent workloads (re-benchmark on your workload) [\[4 peer\]](#).

2. **High-Volume Processing Efficiency:** For enterprise document extraction and high-throughput pipelines, default to single-agent or deterministic state-graph architectures when target accuracy is bounded around 89% [\[12 specialist\]](#). Avoid dynamic multi-agent orchestration unless verified task accuracy gains offset the baseline 1.15x token cost multiplier (re-benchmark on your workload) [\[12 specialist\]](#).
3. **Search and Retrieval Optimization:** For multi-hop search and tool navigation tasks, select step-wise post-training frameworks (such as StepSearch with step-wise PPO) over global-reward RL models [\[2 peer\]](#). Expect potential accuracy gains of +11.2 percentage points on 3B parameter models and +4.2 percentage points on 7B models using dense process supervision datasets (re-benchmark on your workload) [\[2 peer\]](#).
4. **Inter-Agent Communication Protocol Selection:** Use Model Context Protocol (MCP) for host-managed applications where tool authorization and local security constraints are critical [\[9 specialist\]](#). Transition to Agent2Agent (A2A) protocol when deploying decentralized, multi-organization agent federations that require peer discovery and asynchronous task messaging [\[9 specialist\]](#).
5. **Bounding Reflection Mechanics:** When implementing self-reflection mechanisms, enforce a hard limit of 2 to 3 execution turns and require external environment feedback (such as compiler output or test execution) [\[11 specialist\]](#). Self-reflection without external feedback fails to improve accuracy, while external API perturbations can induce benchmark evaluation variability up to 2.8 percentage points (re-benchmark on your workload) [\[11 specialist\]](#).

Uncertainties & gaps

MEASUREMENT GAPS

- **Protocols Wall-Clock Latency Profiles:** Existing literature lacks standardized, wall-clock latency measurements (in milliseconds) comparing inter-agent coordination protocols (MCP vs. A2A) across varied network topologies and serving frameworks [\[9 specialist\]](#).
- **Standardized Serving Expenditure:** Published benchmarks report token counts and relative cost multipliers [\[4 peer\]](#)[\[12 specialist\]](#), but omit standardized dollar-per-request deployment costs across cloud hardware platforms (such as NVIDIA H100 vs. A100 clusters).
- **Compute and FLOP Consumption:** Compute overhead (total floating-point operations) incurred during dynamic test-time scaling, Monte Carlo Tree Search, and step-wise re-planning loops remains unquantified across benchmark literature.

References

1. Agentic AI: a comprehensive survey of architectures, applications, and future directions — <https://doi.org/10.1007/s10462-025-11422-4>
2. StepSearch: Igniting LLMs Search Ability via Step-Wise Proximal Policy Optimization — <https://doi.org/10.18653/v1/2025.emnlp-main.1106>
3. The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges — <https://doi.org/10.3390/fi17090404>
4. The Illusion of Agentic Complexity in README.md Generation: Evaluating Single-Agent vs. Multi-Agent RAG Systems — <https://doi.org/10.48550/arXiv.2606.30524>
5. Complex QA and language models hybrid architectures, Survey — <http://arxiv.org/abs/2302.09051>
6. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks — <http://arxiv.org/abs/2411.04468>
7. Adaptation of Agentic AI: A Survey of Post-Training, Memory, and Skills — <http://arxiv.org/abs/2512.16301>
8. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey — <https://arxiv.org/abs/2509.02547>
9. A Comparative Study of MCP and A2A for Inter-Agent Coordination in LLM-Based Systems — <https://arxiv.org/abs/2607.23884>
10. Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs — <https://arxiv.org/html/2311.17371v3>
11. Self-Reflection in LLM Agents: Effects on Problem-Solving Performance — <https://arxiv.org/html/2405.06682v2>
12. Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies — <https://arxiv.org/html/2603.22651v1>

SOURCES

[1] PEER REVIEWED
Agentic AI: a comprehensive survey of architectures, applications, and future directions

doi.org

[2] PEER REVIEWED

StepSearch: Igniting LLMs Search Ability via Step-Wise Proximal Policy Optimization

[doi.org](#)

[3] PEER REVIEWED

The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges

[doi.org](#)

[4] PEER REVIEWED

The Illusion of Agentic Complexity in README.md Generation: Evaluating Single-Agent vs. Multi-Agent RAG Systems

[doi.org](#)

[5] SPECIALIST

Complex QA and language models hybrid architectures, Survey

[arxiv.org](#)

[6] SPECIALIST

Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks

[arxiv.org](#)

[7] SPECIALIST

Adaptation of Agentic AI: A Survey of Post-Training, Memory, and Skills

[arxiv.org](#)

[8] SPECIALIST

The Landscape of Agentic Reinforcement Learning for LLMs: A Survey

[arxiv.org](#)

[9] SPECIALIST

A Comparative Study of MCP and A2A for Inter-Agent Coordination in LLM-Based Systems

[arxiv.org](#)

[10] SPECIALIST

Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs

[arxiv.org](#)